



ಬಿ.ಎಂ.ಎಸ್. ತಾಂತ್ರಿಕ ಮತ್ತು ವ್ಯವಸ್ಥಾಪನಾ ಮಹಾವಿದ್ಯಾಲಯ

BMS INSTITUTE OF TECHNOLOGY & MANAGEMENT

(Autonomous Institution Under VTU)

Yelahanka, Bengaluru -560119



A Full Stack Mini-project Work Report

on

“VersionIQ: AI-Powered Version Intelligence and DevOps Automation Platform”

Submitted in partial fulfillment for the award of

BACHELOR OF ENGINEERING

in

INFORMATION SCIENCE AND ENGINEERING

by

Ms. Supriya Gouda 1BY23IS227

Ms. Vidhi Poddar 1BY23IS248

Under the Guidance of

Prof. Sowmya K

Assistant Professor

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

Avalahalli, Yelahanka, Bengaluru, Karnataka – 560119

April – 2026



ಬಿ.ಎಂ.ಎಸ್. ತಾಂತ್ರಿಕ ಮತ್ತು ವ್ಯವಸ್ಥಾಪನಾ ಮಹಾವಿದ್ಯಾಲಯ

BMS INSTITUTE OF TECHNOLOGY & MANAGEMENT

(Autonomous Institution Under VTU)

Yelahanka, Bengaluru -560119

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the Full Stack Mini-Project work (BCS604C) entitled “**VersionIQ: AI-Powered Version Intelligence and DevOps Automation Platform**” is a bonafide work carried out by **Ms. Supriya Gouda(1BY23IS227)**, **Ms. Vidhi Poddar (1BY23IS248)** in partial fulfillment for the award of **Bachelor of Engineering Degree in Information Science and Engineering** of the Visvesvaraya Technological University, Belagavi during the year 2025–26. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in this report. The Full stack Mini-project report has been approved as it satisfies the academic requirements with respect to Full Stack Mini-project work for the B.E Degree.

Signature of the
Guide

Signature of the
Cluster Head

Signature of the
HOD, ISE

ABSTRACT

VersionIQ is an AI-powered version intelligence and DevOps automation platform designed to manage file uploads, version history, semantic change analysis, rollback recommendations, and CI/CD workflow monitoring in a single system. The platform allows users to securely upload files, create new versions on every update, and view the complete history of each file. For every version change, the backend reads the actual file content from disk, compares the current version with the previous version, and generates an AI-based summary that explains what changed in a human-readable form.

The application is built using React with TypeScript for the frontend and Node.js with Express.js for the backend. MongoDB is used to store user data, file metadata, version records, summaries, activity logs, and pipeline information, while Multer is used to handle file uploads and store the actual files on the server file system. The system integrates Gemini AI to generate meaningful summaries of version differences and also provides fallback local diff-based summaries when AI service is unavailable.

VersionIQ also includes smart rollback recommendations, allowing users to restore stable previous versions when required. In addition, the project integrates Jenkins and Docker to support automated build, test, and deployment workflows. The DevOps dashboard displays pipeline status, build logs, and deployment health checks, giving users real-time visibility into the CI/CD process. Overall, VersionIQ combines AI, version management, and DevOps automation into a unified platform for intelligent file tracking and deployment monitoring.

INDEX

List of Figures	5
1 INTRODUCTION	6
1.1 Background	6
1.2 Need for Goal Management Systems	6
1.3 Role of Full Stack Development	6
1.4 Limitations of Existing Systems	7
1.5 Proposed Approach	7
2 PROBLEM STATEMENT	9
3 OBJECTIVES	10
4 LITERATURE SURVEY	11
5 RESEARCH GAP AND RESEARCH CHALLENGES	12
6 PROPOSED METHODOLOGY	13
6.1 Approach	13
6.2 System Architecture	13
6.3 Tools and Technologies	14
6.4 Process	16
6.4.1 User Registration and Authentication	16
6.4.2 File Input and Processing	16
6.4.3 AI-Based Semantic Version Analysis	16
6.4.4 Data Storage and Management	17
6.4.5 Version Tracking and Monitoring	17
6.4.6 Rollback Recommendations and AI Assistance	17
6.4.7 Dashboard Analytics and Visualization	17
6.4.8 CI/CD Automation and Deployment Monitoring	17
7 RESULTS	18
8 CONCLUSION	20

LIST OF FIGURES

1	System Architecture of VersionIQ	14
2	Landing Page of VersionIQ	18
3	Workspace Overview Dashboard of VersionIQ Platform	18
4	AI-Powered Summary Analysis Dashboard	19
5	Smart Rollback Recommendation and Version Recovery Interface	19
6	DevOps CI/CD Pipeline Monitoring and Deployment Dashboard	19

1 INTRODUCTION

1.1 Background

Version management plays a critical role in modern software development, document handling, and deployment workflows. In many traditional systems, files are frequently updated or overwritten without maintaining meaningful information about the changes made between versions. Although some systems preserve file history, they often fail to provide a clear understanding of what changed, why the changes were made, and how those changes affect system stability or deployment reliability.

With the rapid growth of Artificial Intelligence and cloud-based DevOps technologies, there is an increasing demand for intelligent systems that can not only manage file versions but also analyze and explain version changes automatically. AI-powered systems can simplify the understanding of modifications by generating semantic summaries and identifying important differences between file versions.

1.2 Need for Goal Management Systems

Managing multiple versions of files manually becomes difficult when files are updated frequently or used in deployment-sensitive environments. Users often struggle to identify what modifications were introduced in newer versions, whether the updates are stable, and which previous version should be restored when failures occur.

Traditional storage systems mainly focus on file preservation rather than intelligent version analysis. Existing systems generally provide raw difference comparisons or line-based modifications that may be difficult for users to understand, especially for large configuration files, scripts, or project documents.

1.3 Role of Full Stack Development

Full stack development involves integrating frontend technologies, backend services, databases, APIs, and deployment systems to create complete web applications. Modern full stack technologies provide scalable, responsive, and efficient solutions for developing intelligent platforms with real-time interaction and automation capabilities.

In VersionIQ, React with TypeScript is used to build the frontend interface for file uploads, version browsing, AI summary visualization, rollback recommendations, and DevOps monitoring dashboards. The backend is developed using Node.js and Express.js to manage authentication, API handling, file processing, AI integration, and CI/CD communication.

MongoDB is used to securely store user information, file metadata, version history, activity logs, summaries, and deployment records. Multer middleware handles secure file uploads and storage operations. Jenkins and Docker are integrated to automate build, test, deployment, and monitoring workflows.

Together, these technologies form a complete full stack architecture capable of supporting AI-powered version intelligence and DevOps automation.

1.4 Limitations of Existing Systems

Many existing file storage and version management systems focus only on preserving versions without providing meaningful explanations of the changes between them. Traditional systems often require users to manually inspect differences line by line, which becomes inefficient for large files and complex deployments.

Most version control systems are designed specifically for source code repositories and may not provide dashboard-driven semantic summaries or AI-based change analysis for general uploaded files. Cloud storage platforms preserve versions but generally lack rollback intelligence, deployment awareness, and integrated CI/CD monitoring capabilities.

Existing systems also do not provide centralized integration of file versioning, AI-powered summaries, rollback recommendations, Docker deployment, and Jenkins pipeline monitoring within a single platform. This creates fragmented workflows where users must depend on multiple separate tools for version management and deployment operations.

1.5 Proposed Approach

The proposed system, VersionIQ: AI-Powered Version Intelligence and DevOps Automation Platform, provides a centralized and intelligent solution for managing file versions, understanding version differences, and automating deployment workflows.

The platform allows users to upload files and automatically maintain complete version history for every update. Whenever a new version is uploaded, the system compares the current version

with the previous version by reading the actual file contents from disk. The extracted content is analyzed using the Gemini AI API to generate semantic summaries that explain the changes in a human-readable format.

The application is developed using React with TypeScript for frontend development and Node.js with Express.js for backend services. MongoDB is used for storing metadata, user information, summaries, activity logs, and pipeline records, while Multer manages secure file uploads and file storage operations.

The system also integrates Docker and Jenkins to support automated CI/CD workflows. Jenkins pipelines automatically build, test, and deploy the application whenever changes are pushed to GitHub. The DevOps dashboard provides real-time monitoring of pipeline execution, deployment status, and build activity.

The proposed approach improves version understanding, deployment visibility, rollback reliability, and overall workflow efficiency by combining Artificial Intelligence, version management, and DevOps automation into a single integrated platform.

2 PROBLEM STATEMENT

In modern software development and file management environments, files are frequently modified, updated, and deployed across multiple systems. Traditional storage systems and version management tools mainly focus on preserving file history but often fail to provide meaningful insights into what changes were made between versions and how those changes impact system stability or deployment reliability.

Users generally rely on manual comparison methods or raw line-by-line differences to understand file modifications. This process becomes difficult and time-consuming for large files, configuration files, deployment scripts, and continuously evolving project assets. Existing systems also lack intelligent rollback recommendations, semantic change explanations, and centralized DevOps monitoring capabilities.

In deployment workflows, even small configuration changes may lead to build failures, unstable deployments, or application downtime. Without proper version intelligence and automated monitoring, identifying the root cause of failures becomes complex and inefficient.

Therefore, there is a need for an intelligent version management platform that can automatically analyze file changes, generate AI-powered summaries, recommend stable rollback versions, and integrate with modern DevOps workflows. The proposed system, VersionIQ, addresses these challenges by combining AI-based semantic analysis, file version tracking, rollback intelligence, Docker deployment, and Jenkins CI/CD automation into a unified full stack platform.

3 OBJECTIVES

1. To develop a secure full stack web application for file upload, storage, and version management.
2. To generate AI-powered semantic summaries that explain the differences between file versions in human-readable form.
3. To implement file comparison and diff analysis for identifying added, removed, and modified content between versions.
4. To develop a rollback recommendation engine that helps users restore stable previous versions.

4 LITERATURE SURVEY

Several research works and modern development practices emphasize the importance of version management, DevOps automation, and Artificial Intelligence in improving software reliability and workflow efficiency.

Research related to AI-assisted code and document summarization demonstrates that large language models can convert technical file differences into meaningful natural language explanations. AI-generated summaries help users quickly understand modifications without manually analyzing raw differences.

Version control systems such as Git and GitHub have established the importance of maintaining version history and tracking changes in software projects. These systems effectively preserve commit history and support collaborative development workflows. However, they mainly focus on source code repositories and do not provide semantic AI explanations for general uploaded files or dashboard-based deployment intelligence.

Studies on DevOps and CI/CD automation highlight how Jenkins and Docker improve deployment consistency, scalability, and operational efficiency. Continuous Integration and Continuous Deployment pipelines automate building, testing, and deployment processes, reducing manual effort and minimizing deployment errors.

Research on intelligent rollback and deployment monitoring systems indicates that automated rollback recommendations and pipeline analytics significantly improve system reliability and reduce downtime during deployment failures.

The literature survey suggests that integrating Artificial Intelligence, version management, rollback analysis, and DevOps monitoring into a unified platform can improve productivity, deployment visibility, and file change understanding. VersionIQ applies these concepts to create an AI-powered version intelligence and DevOps automation platform.

5 RESEARCH GAP AND RESEARCH CHALLENGES

Research Gaps Identified

- Most existing file storage and version management systems mainly preserve file history without providing meaningful semantic explanations of changes between versions.
- Traditional version control platforms focus primarily on source code repositories and lack AI-powered summaries for general uploaded files and deployment configurations.
- Existing systems generally provide raw line-based differences rather than human-readable AI-generated explanations of modifications.
- Many available platforms do not combine version intelligence, rollback recommendations, AI analysis, and DevOps monitoring into a single centralized system.
- Current cloud storage systems preserve versions but lack deployment-aware rollback analysis and CI/CD integration capabilities.

Research Challenges

- Extracting readable content from multiple file formats while handling large files safely and efficiently.
- Generating meaningful semantic summaries instead of generic line-count outputs from AI models.
- Maintaining immutable version history while supporting rollback and restore operations.
- Integrating AI-based file analysis with real-time version management and DevOps workflows. Ensuring secure authentication, user isolation, and protected file access within the platform.
- Designing an intuitive dashboard that combines file versioning, AI summaries, rollback intelligence, and pipeline analytics in a user-friendly manner.

6 PROPOSED METHODOLOGY

6.1 Approach

The proposed VersionIQ: AI-Powered Version Intelligence and DevOps Automation Platform follows an AI-integrated full stack web application approach for intelligent file version management, semantic change analysis, rollback recommendation, and DevOps automation.

The system allows users to securely upload files and maintain complete version history for every update. Whenever a new version is uploaded, the backend compares the current file with the previous version and analyzes the differences using Artificial Intelligence. The AI engine generates semantic summaries that explain the modifications in a human-readable format, helping users quickly understand what changed between versions.

The platform also integrates Jenkins and Docker to automate Continuous Integration and Continuous Deployment workflows. The methodology focuses on improving deployment visibility, reducing manual analysis effort, supporting rollback operations, and simplifying version management through intelligent automation.

6.2 System Architecture

The proposed system follows a client-server architecture using modern full stack technologies with AI and DevOps integration. The architecture is designed to provide secure communication, scalable processing, automated deployment workflows, and centralized monitoring.

- **User (Client):** Users interact with the platform through a web-based interface where they can upload files, manage versions, compare changes, view AI summaries, monitor deployments, and restore previous versions.
- **Frontend (React + TypeScript):** The frontend provides an interactive and responsive user interface. It handles file uploads, version history visualization, semantic diff viewing, rollback recommendations, dashboard analytics, and DevOps monitoring interfaces.
- **Backend Server (Node.js + Express.js):** The backend server manages authentication, API routing, file upload processing, version creation, AI integration, rollback services, and communication with the database and DevOps services.
- **AI Service (Gemini API):** The backend communicates with the Gemini AI API to analyze previous and current file content. The AI service generates semantic summaries that describe added, removed, and modified changes between versions.

- **Database (MongoDB):** MongoDB is used to securely store user information, file meta-data, version history, AI summaries, activity logs, rollback records, and pipeline monitoring details.
- **File Storage System:** Actual uploaded files are stored on the server file system using Multer middleware. The files are stored in structured directories according to user ID, file ID, and version number.
- **CI/CD and Deployment Layer (Jenkins + Docker):** Jenkins automates the CI/CD pipeline including build, test, deployment, and health verification. Docker containerizes the frontend, backend, MongoDB, and Jenkins services for consistent deployment environments.
- **Communication Flow:** The frontend communicates with the backend through RESTful APIs. The backend communicates with MongoDB for metadata storage, Gemini AI for semantic analysis, and Jenkins for pipeline monitoring and deployment synchronization.

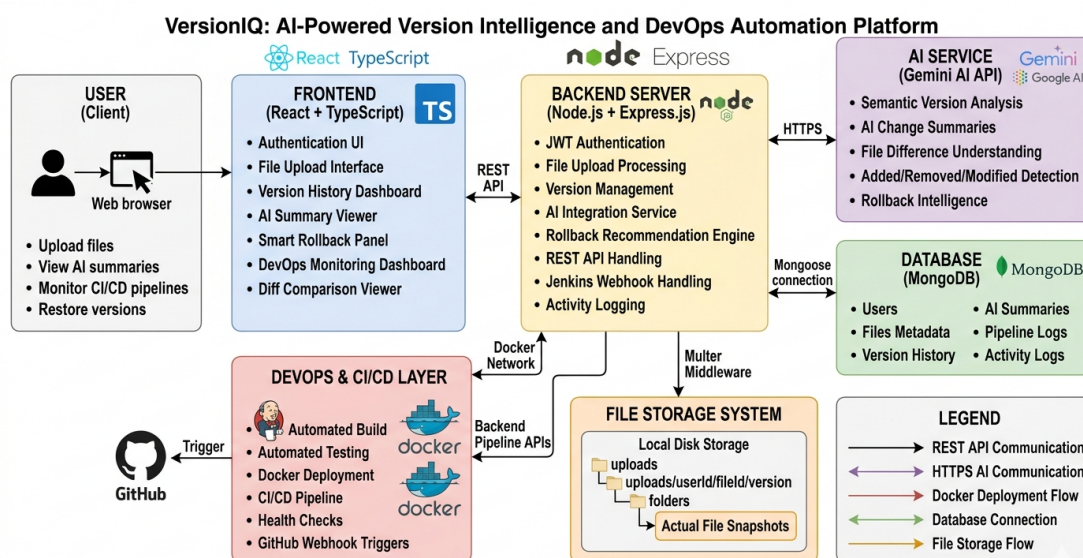


Figure 1: System Architecture of VersionIQ – AI-Powered Version Intelligence and DevOps Semantic Platform

Figure 1: System Architecture of VersionIQ

6.3 Tools and Technologies

To build the proposed Meridian: AI-Powered Smart Goal Decomposer and Progress Tracking System, the following tools and technologies are used:

Technological Stack:

Frontend Development-

- React.js (Interactive and dynamic user interface)
- TypeScript (Type-safe frontend development)
- Vite (Fast frontend build tool and development server)
- Tailwind CSS (Responsive and modern UI styling)

Backend Development-

- Node.js (Server-side runtime environment)
- Express.js (RESTful API and backend service handling)
- JWT Authentication (Secure user authentication and authorization)
- bcryptjs (Password hashing and security)
- Multer (File upload handling middleware)

Database-

- MongoDB (NoSQL database for storing metadata and user information)
- Mongoose (MongoDB object modeling and schema management)

Artificial Intelligence and DevOps -

- Google Gemini API (AI-powered semantic file analysis)
- Docker (Containerization and deployment)
- Jenkins (CI/CD automation and pipeline management)
- GitHub (Version control and repository hosting)

Development tools -

- Antigravity (Code editor and development environment)
- Git and GitHub (Version control and project management)
- Postman (API testing and debugging)
- Docker Desktop (Container management)

6.4 Process

The working process of VersionIQ begins with user registration and secure login. After authentication, users can upload files through the frontend interface. The uploaded file is processed by the backend server using Multer middleware and stored securely on the server file system.

If the uploaded file already exists, the system automatically creates a new version instead of overwriting the previous file. The backend reads the current file and the previous version, compares their content, and generates semantic summaries using Gemini AI.

The generated summaries, metadata, rollback records, activity logs, and pipeline details are stored in MongoDB. Users can view version history, compare changes, inspect AI summaries, restore stable versions, and monitor CI/CD pipelines through the dashboard.

The Jenkins pipeline continuously monitors GitHub repository updates and automatically performs build, test, deployment, and health verification operations using Docker containers.

6.4.1 User Registration and Authentication

The process begins with secure user registration and login. Users create accounts using email and password credentials. JWT-based authentication is implemented to maintain secure sessions and protect access to authorized resources.

The authentication system validates user credentials, generates tokens, and ensures that uploaded files and version histories remain user-specific and secure.

6.4.2 File Input and Processing

After authentication, users upload files through the frontend interface. The frontend sends the file data to the backend using RESTful API requests. The backend receives the uploaded file using Multer middleware, validates the file details, and stores the file in structured upload directories. If the file already exists, the system prepares a new version entry instead of replacing the old version.

6.4.3 AI-Based Semantic Version Analysis

The backend extracts readable content from both the current version and the previous version of the uploaded file. The extracted content is sent to the Gemini AI API for semantic comparison.

The AI engine analyzes the differences between versions and generates human-readable summaries describing added, removed, and modified changes. The AI system also generates meaningful explanations rather than only line-based differences.

6.4.4 Data Storage and Management

MongoDB is used to securely store metadata related to uploaded files, users, versions, summaries, rollback records, activity logs, and deployment pipelines.

The actual uploaded files are stored separately on the server file system. Mongoose is used for schema modeling, validation, and efficient database operations.

6.4.5 Version Tracking and Monitoring

The system continuously tracks every uploaded file version and maintains immutable version history. Users can monitor file evolution, compare versions, inspect AI summaries, and identify deployment-related modifications.

The platform also tracks activity logs, deployment status, and rollback history to provide complete visibility into file and deployment operations.

6.4.6 Rollback Recommendations and AI Assistance

The system analyzes version stability and provides rollback recommendations when deployment failures or unstable updates occur.

AI-generated summaries and semantic analysis help users identify risky modifications and understand the impact of changes before restoring previous versions.

6.4.7 Dashboard Analytics and Visualization

The application provides interactive dashboards and graphical analytics for version tracking, deployment monitoring, AI summaries, rollback suggestions, and CI/CD pipeline activity.

Users can visualize version history, compare semantic changes, monitor build status, and inspect deployment logs through centralized dashboard interfaces.

6.4.8 CI/CD Automation and Deployment Monitoring

The Jenkins pipeline continuously monitors GitHub repository updates and automatically triggers build, testing, deployment, and health verification workflows.

Docker containerization ensures consistent execution environments for frontend, backend, MongoDB, and Jenkins services. The DevOps dashboard displays real-time deployment logs, pipeline activity, and container health monitoring information.

7 RESULTS

The following are the major results and outcomes obtained from the implementation of the proposed VersionIQ: AI-Powered Version Intelligence and DevOps Automation Platform. The system successfully demonstrates intelligent file version management, AI-powered semantic change analysis, rollback recommendations, CI/CD automation, deployment monitoring, and secure full-stack web application functionality.

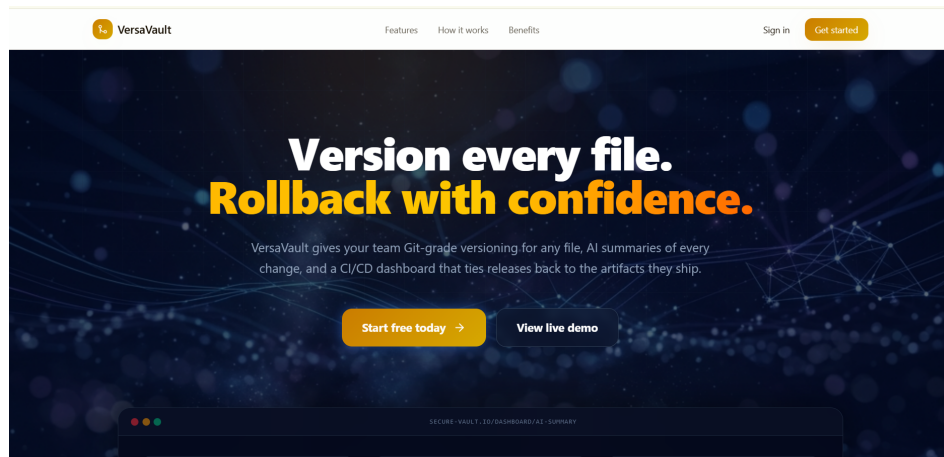


Figure 2: Landing Page of VersionIQ

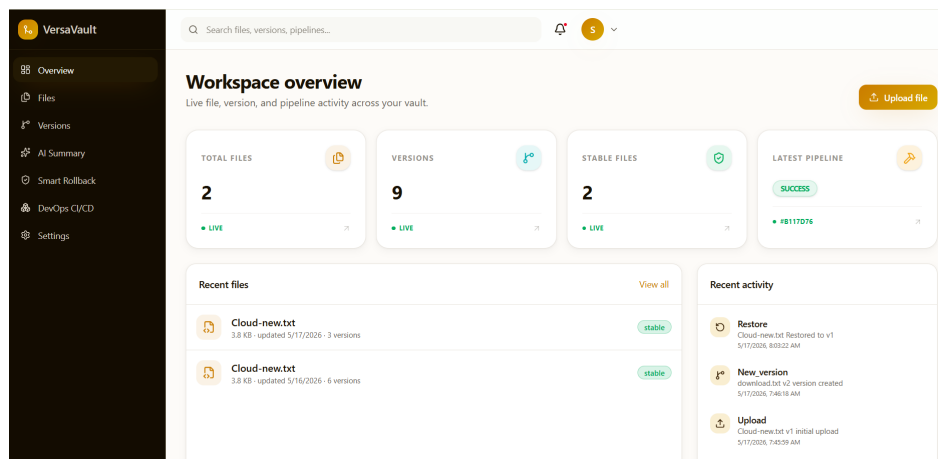


Figure 3: Workspace Overview Dashboard of VersionIQ Platform

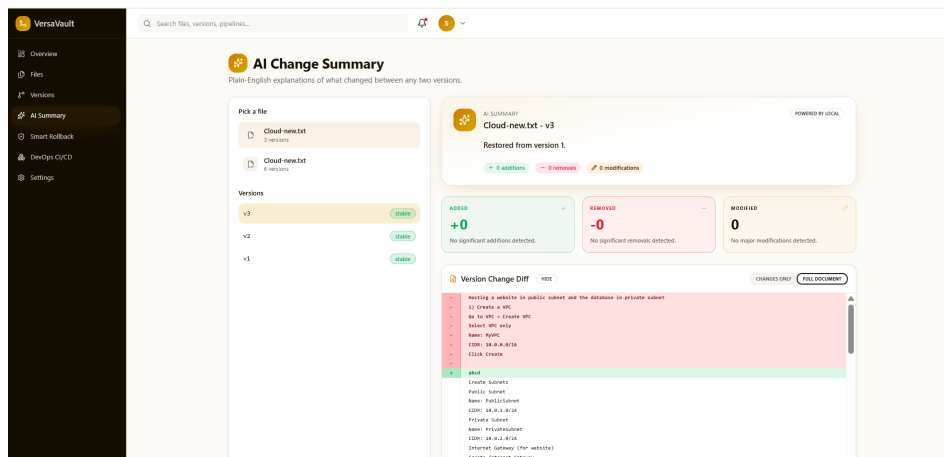


Figure 4: AI-Powered Summary Analysis Dashboard

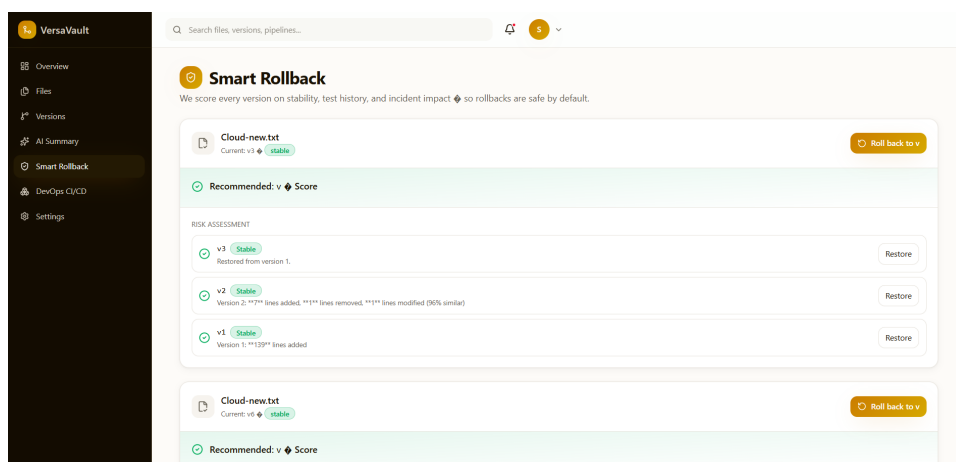


Figure 5: Smart Rollback Recommendation and Version Recovery Interface

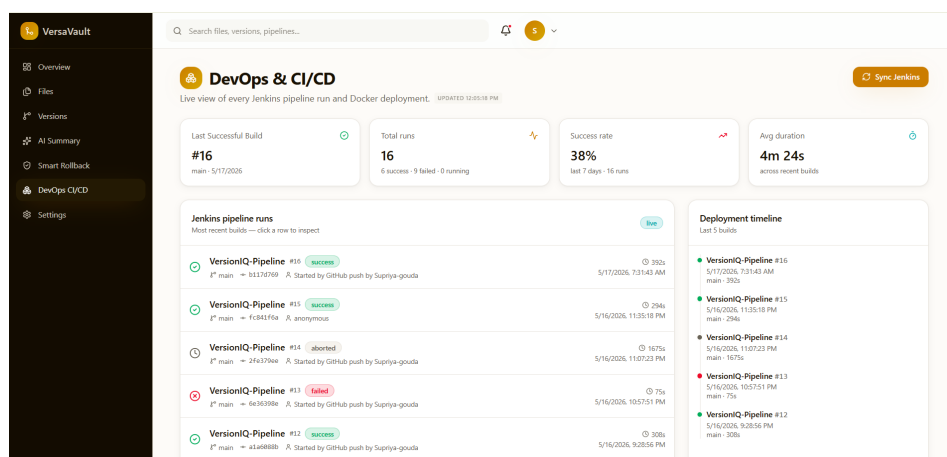


Figure 6: DevOps CI/CD Pipeline Monitoring and Deployment Dashboard

8 CONCLUSION

VersionIQ: AI-Powered Version Intelligence and DevOps Automation Platform provides an intelligent and efficient solution for file version management, semantic change analysis, rollback support, and CI/CD workflow automation. The system successfully combines Artificial Intelligence with modern full stack technologies to simplify version tracking, improve deployment visibility, and reduce manual effort involved in analyzing file modifications.

The integration of Gemini AI enables the platform to generate meaningful semantic summaries between file versions, helping users understand the actual impact of changes rather than relying only on raw line-based differences. The rollback recommendation system further improves reliability by assisting users in restoring stable versions during deployment failures or unstable updates.

The use of React with TypeScript, Node.js, Express.js, MongoDB, Multer, Docker, and Jenkins provides a scalable and robust architecture capable of handling secure authentication, file processing, AI integration, version comparison, deployment automation, and centralized monitoring.

The Jenkins CI/CD integration and Docker containerization automate build, testing, and deployment workflows, improving operational consistency and deployment efficiency. The DevOps dashboard provides real-time monitoring of pipeline activity, deployment logs, and container health, enabling better visibility into system operations.

Overall, the proposed system demonstrates how Artificial Intelligence, full stack development, and DevOps automation can be effectively integrated to build a modern version intelligence platform capable of improving file management, deployment reliability, and workflow productivity.